YENEPOYA INSTITUTE OF ARTS, SCIENCE, COMMERCE AND MANAGEMENT

Project – High Level Design

On

Cloud Cost Intelligence Platform for Supply Chain

SUBMITTED BY

Muhammad Rijaz Schemnad - 23BCAICD067

BCA(AI, Cloud Computing, DevOps)

UNDER THE GUIDANCE OF

Industry Mentor: Shashank Korrram

Guided by: Sindhu

TABLE OF CONTENTS

1. Introduction……02
2. Scope of the Document……..03
3. Intended Audience……..04
4. System Overview……....05
5. System Design……07
5.1 Application Design……07
5.2 Process Flow……...08
5.3 Information Flow……10
5.4 Components Design……11
5.5 Key Design Considerations……13
6. API Catalogue…….....14
7. Data Design……....15
7.1 Data Model…….....15
7.2 Data Access Mechanism……16
7.3 Data Retention Policies…….....17
7.4 Data Migration……....17
8. Interfaces…….....18
9. State and Session Management……...18
10. Caching…….....19
11. Non-Functional Requirements……19
11.1 Security Aspects……..19
11.2 Performance Aspects…….....19
12. References……...20

1. Introduction

The Cloud Cost Intelligence Platform for Supply Chain is a comprehensive, end-to-end cloud cost management and analytics system. It is designed to help organizations operating in the supply chain domain to understand, analyze, and optimize their cloud infrastructure spending across multiple cloud providers (AWS, Azure, GCP, Oracle Cloud).

In today's enterprise environment, supply chain operations rely heavily on cloud infrastructure for warehousing systems, transport logistics, order processing, and delivery tracking. Each of these operations generates cloud billing data across various services such as compute, storage, database, networking, machine learning, analytics, serverless, and containers. Without a centralized intelligence platform, organizations struggle to identify cost drivers, detect inefficiencies, and make informed cost optimization decisions.

This High-Level Design (HLD) document provides an architectural overview of the entire platform, with a particular focus on:

- The Cost Calculator Engine — how cloud costs are computed, aggregated, and analyzed
- The Web-Based Cost Intelligence Dashboard — an interactive platform with real-time filtering by timeframe (7/30/90 days), cloud provider, service type, and warehouse location
- Visualization System — Chart.js powered charts including cost trends, provider distribution, service breakdown, and warehouse comparison

- Multi-Cloud Cost Comparison — side-by-side analysis of AWS, Azure, GCP, and Oracle Cloud spending
- Excel Report Generation — professional multi-sheet billing reports for stakeholder distribution
- Alert & Recommendation Engine — automated anomaly detection and actionable optimization suggestions

The platform is built as a two-tier system: a Python-based data processing backend (Jupyter Notebooks) and a premium web-based frontend dashboard (HTML/CSS/JavaScript). Data flows from raw cloud billing spreadsheets through a cleaning and enrichment pipeline, producing actionable insights consumed by both the notebook and the web dashboard.

2. Scope of the Document

This HLD document describes the high-level architecture, design patterns, and system components of the Cloud Cost Intelligence Platform. It serves as a blueprint for understanding:

- How the platform is architecturally organized across its processing, storage, and presentation layers
- How the Cost Calculator Engine computes total cost, average cost per unit, cost per order, and efficiency scores
- How the Web Dashboard applies interactive filters and renders real-time visualizations
- How data flows from raw billing spreadsheets to the final interactive dashboard

- How the alert system detects cost anomalies using statistical thresholds
- How recommendations are generated based on detected patterns and inefficiencies
- How the Excel-based reporting system structures multi-sheet professional billing reports

Document Boundaries:

This HLD provides a system-level architectural view. For implementation-level details such as specific data cleaning algorithms (IQR outlier detection, missing value strategies), column-level transformations, and code-level component specifications, refer to the Low-Level Design (LLD) document.

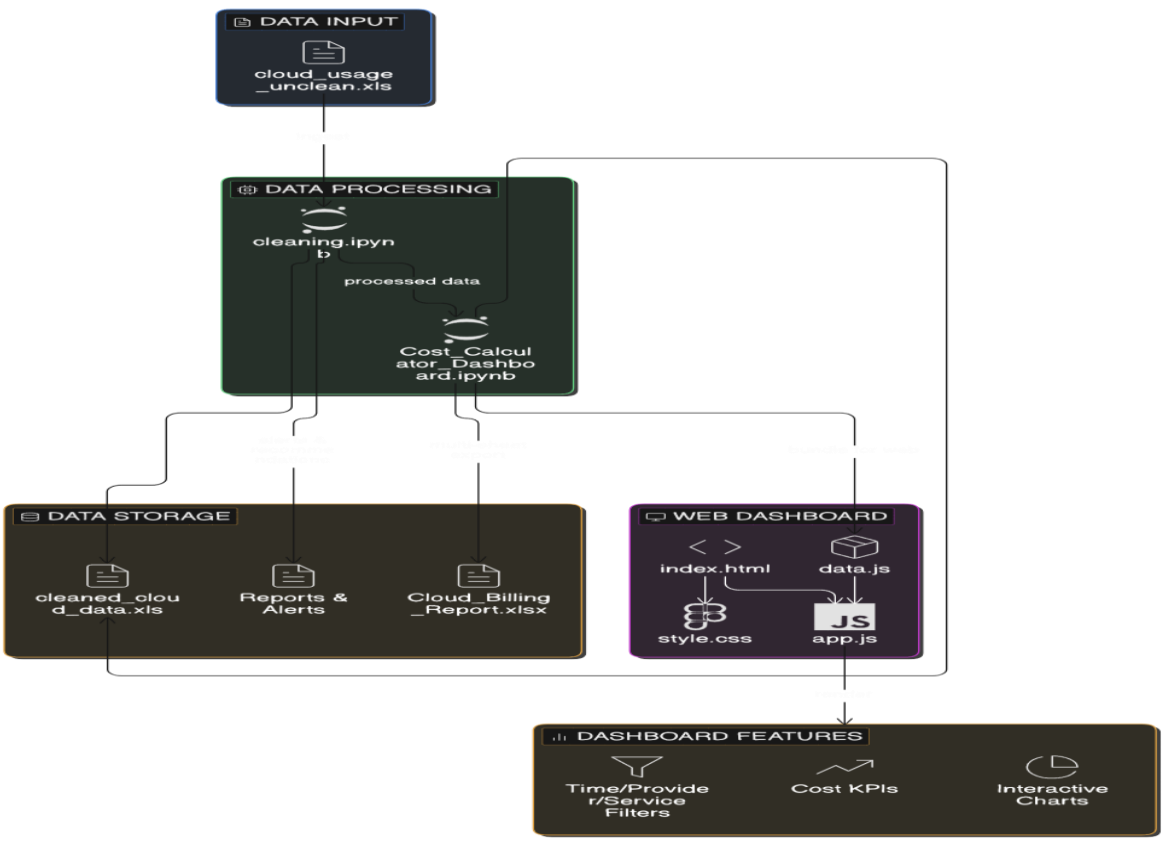
3. Intended Audience

- Project Evaluators / Faculty — Purpose: To evaluate the architectural design and system-level thinking of the project
- Industry Mentors — Purpose: To assess the design against real-world FinOps practices and cloud cost management standards
- Project Managers — Purpose: To understand the scope, modules, and integration points of the platform
- Developers / Future Contributors — Purpose: To understand the overall architecture before diving into implementation details
- Business Stakeholders / FinOps Teams — Purpose: To understand how the dashboard and reports serve cost optimization goals
- Students / Peers — Purpose: To use as an architectural reference for similar cloud analytics projects

4. System Overview

The Cloud Cost Intelligence Platform is structured as a three-layer architecture with clear separation of concerns between data processing, data storage, and data presentation.

4.1 High-Level Architecture:



4.2 Platform Components Summary:

- Input — Component: cloud_usage_unclean.xls, Technology: Excel/CSV Spreadsheet, Purpose: Raw billing data source from supply chain cloud operations
- Processing — Component: cleaning.ipynb, Technology: Python, Pandas, NumPy, Matplotlib, Seaborn, Purpose: Data cleaning, feature engineering, EDA, alert generation, recommendations
- Processing — Component: Cost_Calculator_Dashboard.ipynb, Technology: Python, Pandas, Matplotlib, openpyxl, Purpose: Cost aggregation, pie chart generation, Excel report export, in-notebook HTML dashboard

- Storage — Component: Results/ directory, Technology: XLS, CSV, TXT files, Purpose: Persisted cleaned data, reports, alerts, and distribution summaries
- Presentation — Component: WebPlatform/ (index.html, style.css, app.js, data.js), Technology: HTML5, CSS3, JavaScript ES6, Chart.js, Google Fonts (Inter), Purpose: Interactive web-based cost intelligence dashboard with real-time filtering

4.3 Key Business Metrics Computed:

- Total Cloud Cost — Formula: SUM(total_cost) for filtered records, Business Significance: Overall cloud spending across the selected scope
- Average Cost per Unit — Formula: SUM(total_cost) / SUM(usage_quantity), Business Significance: Cost efficiency of resource utilization
- Cost per 1,000 Units — Formula: Average Cost per Unit x 1000, Business Significance: Standardized cost metric for comparison across services
- Cost per Order — Formula: total_cost / (order_count + 1), Business Significance: Cloud cost attributed to each business transaction
- Efficiency Score — Formula: usage_quantity / (total_cost + 1e-6), Business Significance: Resource utilization efficiency (higher = more efficient)
- Service Cost % — Formula: (service_cost / total_cost) x 100, Business Significance: Percentage contribution of each service to total spend
- Monthly Cost Trend — Formula: GROUP BY month -> SUM(total_cost), Business Significance: Time-series view of spending for trend detection

- Provider Cost Distribution — Formula: GROUP BY provider -> SUM(total_cost), Business Significance: Cost allocation across cloud vendors
- Cost Spike % — Formula: month-over-month percentage change, Business Significance: Detection of abnormal spending increases

5. System Design

5.1 Application Design:

The application is designed around three major subsystems that work together to deliver end-to-end cloud cost intelligence:

Subsystem 1: Data Processing Pipeline:

A sequential, 6-section Jupyter Notebook pipeline (cleaning.ipynb) that transforms raw billing data into cleaned, enriched, and analyzed data. This subsystem handles data ingestion, cleaning, feature engineering, exploratory analysis, cost intelligence with automated alerts, and recommendation generation.

Subsystem 2: Cost Calculator & Report Generator:

The Cost Calculator Dashboard (Cost_Calculator_Dashboard.ipynb) consumes the cleaned dataset and performs cost aggregation by service type and month. It generates professional visualizations (pie charts showing "where the money went") and exports a multi-sheet Excel report containing raw billing data, cost by service summary, and cost by month summary. Additionally, it renders an inline HTML dashboard within the notebook itself using glassmorphism design with computed metrics (Total Cloud Cost, Top Cost Driver, Avg Cost Per Unit).

Subsystem 3: Web-Based Cost Intelligence Dashboard

The Web Platform (WebPlatform/) is an interactive, single-page application (SPA) that provides stakeholders with a premium visual interface for exploring cloud cost data. It features a sidebar with 4 filter controls and a main dashboard area with 3 KPI metric cards and 4 interactive Chart.js charts. The dashboard operates entirely offline using a pre-bundled JavaScript data file, requiring no backend server or internet connection.

Module Interaction Overview:

cleaning.ipynb — Consumes From: Raw XLS file, Produces For: Cleaned XLS + TXT reports, Key Function: Data pipeline: clean, enrich, analyze, alert, recommend

Cost_Calculator_Dashboard.ipynb — Consumes From: Cleaned XLS (or raw fallback), Produces For: Excel report + In-notebook dashboard, Key Function: Cost aggregation, pie charts, Excel export, HTML dashboard

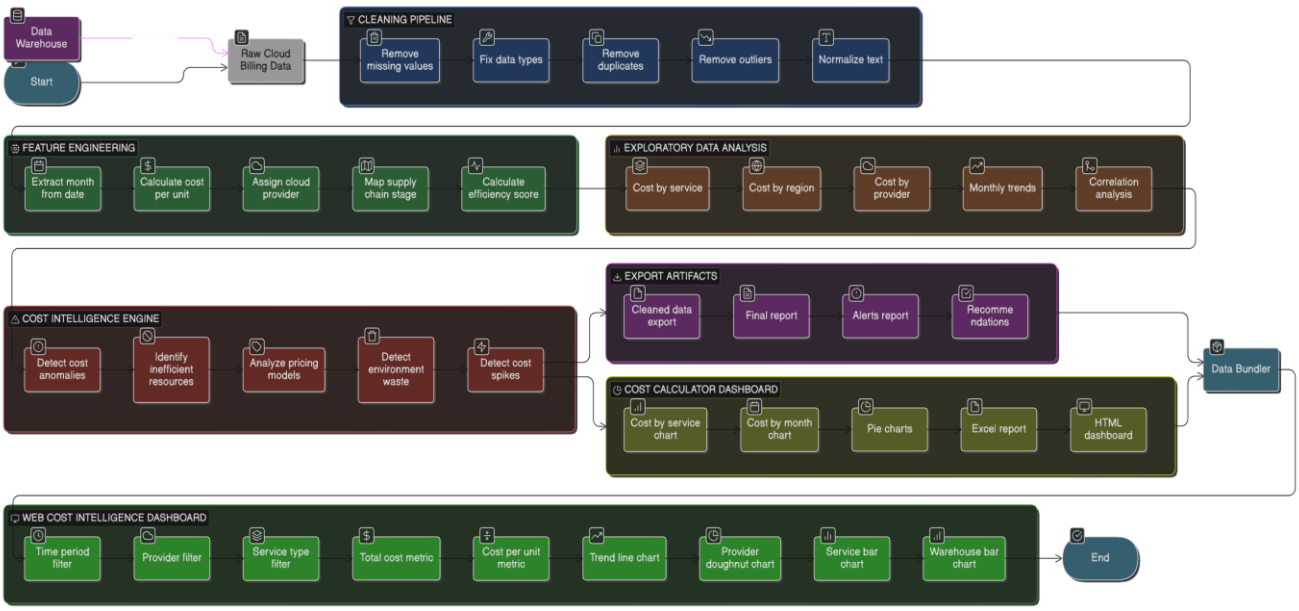
data.js bundler — Consumes From: Cleaned XLS/CSV, Produces For: data.js (JavaScript array), Key Function: Converts tabular data to browser-consumable format

WebPlatform (app.js) — Consumes From: data.js, Produces For: Interactive dashboard in browser, Key Function: Client-side filtering, metric computation, chart rendering

5.2 Process Flow

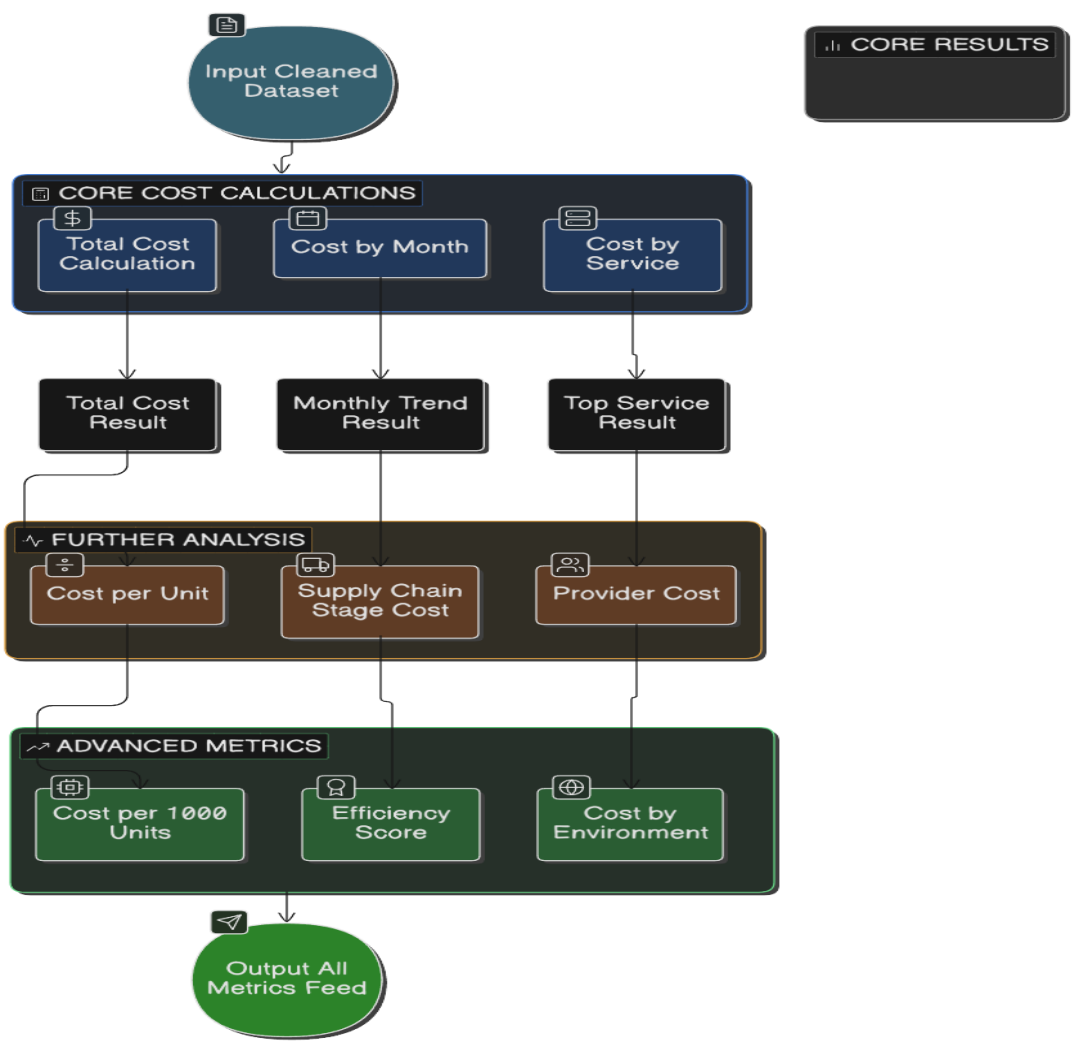
5.2.1 End-to-End System Process Flow:

The following flowchart shows the complete end-to-end flow from raw data input to the final interactive web dashboard that stakeholders use for cost analysis:



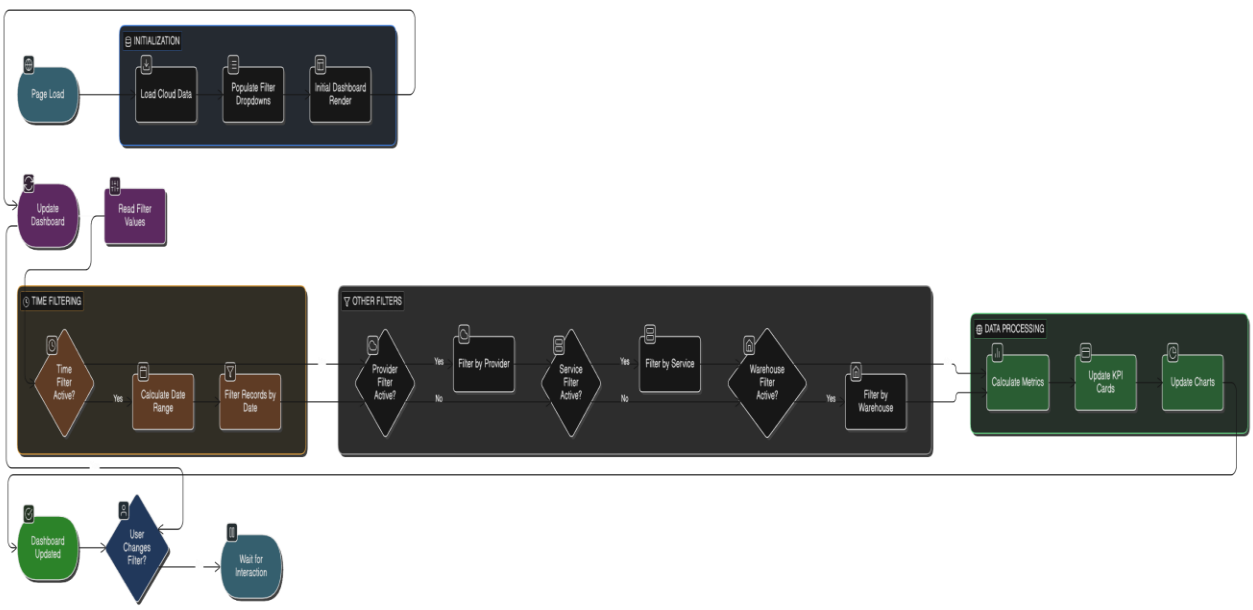
5.2.2 Cost Calculator Engine Process Flow:

The Cost Calculator Engine is the analytical core of the platform. It computes multiple cost metrics from the cleaned dataset. The following flowchart shows how costs are calculated:



5.2.3 Web Dashboard Filter & Rendering Flow:

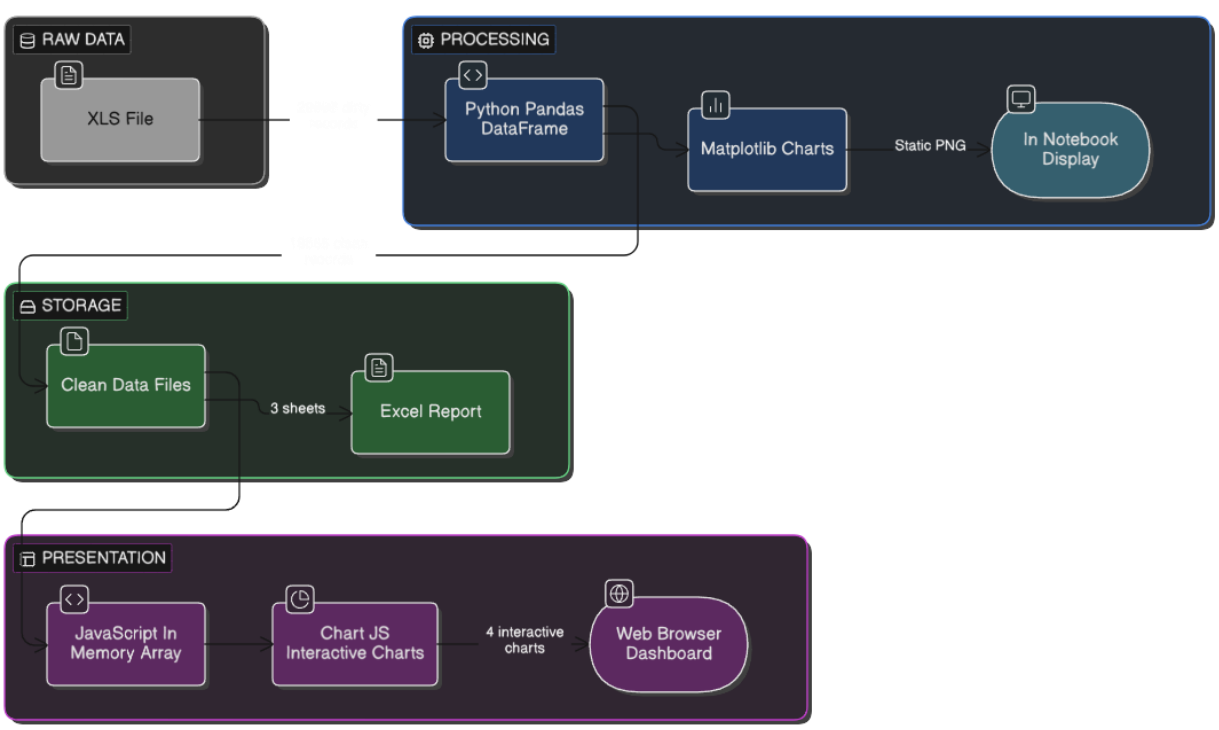
The web dashboard uses a client-side filtering architecture where all filtering, metric computation, and chart rendering happens in the browser. The following flowchart shows the complete interaction flow:



5.3 Information Flow:

The information flow tracks how data is transformed as it moves through the system layers. Each layer enriches the data with additional context and analytical value.

High-Level Data Flow:



5.4 Components Design:

Component 1: Data Processing Pipeline:

Purpose — Detail: Transform raw billing data into clean, enriched, analyzed dataset

Technology — Detail: Python 3.10+, Pandas, NumPy, Matplotlib, Seaborn

Input — Detail: cloud_usage_unclean.xls (~20,000+ records)

Output — Detail: cleaned_cloud_data.xls (18,568 records), reports, alerts, recommendations

Processing Stages — Detail: 6 sequential sections: Ingestion -> Cleaning -> Feature Eng. -> EDA -> Intelligence -> Recommendations

Component 2: Cost Calculator Engine:

- Purpose — Detail: Aggregate costs by service type and month, generate visualizations and Excel reports
- Technology — Detail: Python, Pandas, Matplotlib, openpyxl
- Input — Detail: Cleaned dataset (from cleaning.ipynb or directly from raw data as fallback)
- Output — Detail: Cloud_Billing_Report.xlsx (3 sheets: Raw, Cost by Service, Cost by Month), Pie chart, HTML dashboard
- Key Computations — Detail: cost_by_service = GROUP BY service -> SUM(total_cost), cost_by_month = GROUP BY month -> SUM(total_cost)
- Excel Report Structure — Detail: Sheet 1: Raw Cloud Billing (all records), Sheet 2: Cost by Service (aggregated), Sheet 3: Cost by Month (aggregated)

Component 3: Web Cost Intelligence Dashboard:

- Purpose — Detail: Interactive, real-time cost exploration platform for stakeholders

- Technology — Detail: HTML5, CSS3 (Glassmorphism), JavaScript ES6, Chart.js, Google Fonts (Inter)
- Architecture — Detail: Single-Page Application (SPA) with offline data (no backend server required)
- Data Source — Detail: data.js — pre-bundled JavaScript array containing full dataset
- Filters — Detail: 4 dropdown filters: Timeframe (All/7/30/90 days), Provider, Service, Warehouse
- KPI Metrics — Detail: 3 real-time metric cards: Total Filtered Cost, Cost per 1K Units, Top Expense Service
- Charts — Detail: 4 Chart.js charts: Cost Trend (Line), Provider Distribution (Doughnut), Service Deep Dive (Bar), Warehouse Cost (Bar)
- Design System — Detail: Dark theme with glassmorphism panels, Inter font, CSS custom properties, hover animations

Component 4: Alert & Recommendation Engine:

- Purpose — Detail: Automatically detect anomalies and generate actionable optimization recommendations
- Alert Logic — Detail: Statistical thresholds: mean + 2 x standard deviation for cost and cost_per_unit
- Spike Detection — Detail: Monthly percentage change > mean(pct_change) + 2 x std(pct_change)
- Recommendation Rules — Detail: 1) Optimize highest-cost service, 2) Switch on-demand to reserved/spot, 3) Reduce dev environment cost, 4) Monitor cost spikes
- Output Files — Detail: alerts_report.txt (anomaly alerts), recommendations.txt (optimization actions), final_report.txt (comprehensive report)

5.5 Key Design Considerations:

- Architecture Pattern — Choice Made: Layered (Input -> Processing -> Storage -> Presentation), Rationale: Clean separation of concerns, each layer independently testable and replaceable
- Dashboard Type — Choice Made: Offline Single-Page Application (SPA), Rationale: No server dependency, instant load, works without internet, zero deployment complexity
- Data Bundling Strategy — Choice Made: Pre-bundle data as JavaScript array (data.js), Rationale: Eliminates AJAX/fetch calls, no CORS issues, enables true offline operation
- Filter Implementation — Choice Made: Client-side Array.filter() on in-memory data, Rationale: Sub-millisecond response time, no server round-trips, smooth user experience
- Chart Library — Choice Made: Chart.js (via CDN), Rationale: Lightweight, responsive, canvas-based, supports line/bar/doughnut charts with animations
- CSS Design System — Choice Made: Glassmorphism with dark theme, Rationale: Premium, modern aesthetic appropriate for a FinOps/analytics dashboard
- Time Filter Logic — Choice Made: Rolling window from dataset max date, Rationale: Adapts to any dataset timeframe automatically, not dependent on current date
- Multi-Cloud Support — Choice Made: Region-based provider assignment, Rationale: Simulates realistic enterprise multi-cloud allocation for cost comparison
- Report Format — Choice Made: Multi-sheet Excel (openpyxl), Rationale: Industry-standard format for enterprise financial reporting and stakeholder distribution

6. API Catalogue

The platform uses function-level interfaces rather than REST APIs. The key interfaces are organized by subsystem:

6.1 Data Processing Functions (cleaning.ipynb):

- assign_provider(row) — Input: DataFrame row with region, Output: String: AWS/Azure/GCP/Oracle Cloud, Description: Region-based cloud provider assignment
- map_stage(row) — Input: DataFrame row, Output: String: warehouse/transport/delivery/order_processing/other, Description: Supply chain stage classification
- pd.read_csv(path) — Input: File path string, Output: DataFrame, Description: Load raw billing data
- df.to_csv(path) — Input: DataFrame + path, Output: CSV file, Description: Export cleaned data

6.2 Cost Calculator Functions (Dashboard Notebook):

- df.groupby("service")["total_cost"].sum() — Input: Cleaned DataFrame, Output: Series: service -> cost, Description: Aggregate total cost by service type
- df.groupby("month")["total_cost"].sum() — Input: Cleaned DataFrame, Output: Series: month -> cost, Description: Aggregate total cost by month
- pd.ExcelWriter(path) — Input: DataFrame + path, Output: XLSX file (3 sheets), Description: Export professional multi-sheet Excel report
- cost_by_service.plot(kind="pie") — Input: Aggregated Series, Output: PNG pie chart, Description: Visualize cost distribution by service

6.3 Web Dashboard Functions (app.js):

- updateDashboard() — Trigger: Any filter <select> change or Reset button click, Description: Master function: reads filters, filters data, computes metrics, renders charts
- calculateMetrics(data) — Trigger: Called by updateDashboard(), Description: Computes totalCost, avgCostPer1K, topService from filtered array
- updateCharts(data) — Trigger: Called by updateDashboard(), Description: Destroys old Chart.js instances, groups data, creates 4 new charts
- DOMContentLoaded listener — Trigger: Page load, Description: Initializes filters, populates dropdowns, renders initial dashboard

7. Data Design

7.1 Data Model:

The platform operates on a single primary dataset that undergoes progressive transformation. The high-level data model consists of:

Core Data Fields:

- date — Type: datetime, Source: Raw data (auto-detected), Used By: Time filtering, monthly trends, spark detection
- total_cost — Type: float, Source: Raw data (auto-detected), Used By: All cost calculations, metrics, charts
- service — Type: string, Source: Raw data, Used By: Service filter, service bar chart, cost breakdown
- region — Type: string, Source: Raw data, Used By: Provider assignment, regional analysis
- usage_quantity — Type: float, Source: Raw data, Used By: Cost per unit calculation, efficiency scoring

- pricing_model — Type: string, Source: Raw data, Used By: Pricing optimization analysis
- environment — Type: string, Source: Raw data, Used By: Environment waste detection
- warehouse_location — Type: string, Source: Raw data, Used By: Warehouse filter, warehouse bar chart
- cloud_provider — Type: string, Source: Derived (by region), Used By: Provider filter, provider doughnut chart
- month — Type: Period, Source: Derived (from date), Used By: Monthly aggregation, trend charts
- cost_per_unit — Type: float, Source: Derived (cost/usage), Used By: Efficiency analysis, dashboard avg cost metric
- supply_chain_stage — Type: string, Source: Derived (rule-based), Used By: Supply chain cost attribution

7.2 Data Access Mechanism:

- Python Notebooks — Access Pattern: Read raw data, write results, Mechanism: Pandas read_csv/to_csv, open() for text files
- Cost Calculator — Access Pattern: Read cleaned data, write Excel, Mechanism: Pandas ExcelWriter with openpyxl engine
- Web Dashboard — Access Pattern: Read entire dataset into memory, Mechanism: JavaScript <script> tag loads data.js on page load
- Dashboard Filtering — Access Pattern: Query in-memory array, Mechanism: Array.prototype.filter() with predicate functions
- Dashboard Charts — Access Pattern: Aggregate filtered data, Mechanism: JavaScript reduce/forEach to group by dimensions

7.3 Data Retention Policies:

- Raw Input Data — Retention: Permanent (read-only source of truth), Location: Data/cloud_usage_unclean.xls
- Cleaned Data — Retention: Overwritten on each pipeline run, Location: Results/cleaned_cloud_data.xls
- Versioned Backup — Retention: Persisted as checkpoint, Location: Results/cleaned_cloud_data_v1.xls
- Reports & Alerts — Retention: Overwritten on each run, Location: Results/*.txt
- Excel Report — Retention: Overwritten on each run, Location: Results/Cloud_Billing_Report.xlsx
- Web Data Bundle — Retention: Regenerated from cleaned data, Location: WebPlatform/js/data.js

7.4 Data Migration:

Data migration occurs through the sequential pipeline execution:

1 — Source: Raw XLS, Destination: In-memory DataFrame, Transformation: File I/O load

2 — Source: Raw DataFrame, Destination: Cleaned DataFrame, Transformation: Cleaning pipeline (missing values, types, duplicates, outliers)

3 — Source: Cleaned DataFrame, Destination: Enriched DataFrame, Transformation: Feature engineering (+6 columns)

4 — Source: Enriched DataFrame, Destination: Results/ files, Transformation: Export to XLS, CSV, TXT

5 — Source: Enriched DataFrame, Destination: Aggregated summaries, Transformation: groupby/sum for service, month, provider

6 — Source: Aggregated data, Destination: Excel Report (3 sheets), Transformation: ExcelWriter multi-sheet export

8. Interfaces

8.1 Web Dashboard Interface Layout:

The web dashboard follows a sidebar + main content layout pattern with 4 distinct interface zones:

9. State and Session Management

9.1 Notebook Session:

The Python notebooks maintain state through the Jupyter kernel session. Variables persist in memory during the session, and sequential cell execution ensures correct data flow. Restarting the kernel requires re-executing all cells from the beginning.

9.2 Web Dashboard Session

- The web dashboard manages state entirely on the client side:
- cloudData — Type: JavaScript Array, Lifecycle: Loaded once on page load, immutable, Purpose: Complete dataset — source of truth for all filtering

10. Caching

Python — Caching Mechanism: In-memory DataFrame, Description: Data loaded once into RAM, all operations performed in memory

Python — Caching Mechanism: Versioned checkpoint file, Description: cleaned_cloud_data_v1.xls avoids re-running cleaning pipeline

Python — Caching Mechanism: df_raw = df.copy(), Description: Caches original data for comparison without re-reading file

Web — Caching Mechanism: Pre-bundled data.js, Description: Complete dataset bundled as JavaScript, zero network latency

Web — Caching Mechanism: Browser cache, Description: Static HTML/CSS/JS assets cached by browser

Web — Caching Mechanism: CDN caching, Description: Chart.js and Google Fonts cached via CDN HTTP headers

11. Non-Functional Requirements

11.1 Security Aspects:

- Fully offline system → ensures **data privacy** (no cloud usage)
- No external APIs → **no API key risks**
- Uses OS-level **file permissions** for data protection
- Prevents XSS using **Chart.js canvas rendering**
- No authentication needed → **single-user system**
- No backend/server → **eliminates server-side vulnerabilities**

11.2 Performance Aspects:

- Fast processing → **<30 seconds** for ~20K records
- Quick dashboard load → **<2 seconds** (no server requests)
- Instant filtering → **<10 ms response time**
- Smooth charts → **60 FPS rendering**
- Efficient memory usage → **no memory leaks**

12. References

- 1 — Reference: Pandas Documentation — pandas.pydata.org, Description: Data manipulation and analysis library
- 2 — Reference: NumPy Documentation — numpy.org, Description: Numerical computing library
- 3 — Reference: Matplotlib Documentation — matplotlib.org, Description: Static visualization library
- 4 — Reference: Seaborn Documentation — seaborn.pydata.org, Description: Statistical data visualization
- 5 — Reference: Chart.js Documentation — chartjs.org, Description: Interactive JavaScript charting library
- 6 — Reference: FinOps Foundation — finops.org, Description: Cloud financial management framework
- 7 — Reference: AWS Well-Architected Framework — Cost Optimization, Description: Cloud cost optimization best practices
- 8 — Reference: Jupyter Notebook Documentation, Description: Interactive development environment
- 9 — Reference: openpyxl Documentation, Description: Excel file creation library for Python
- 10 — Reference: Google Fonts — Inter, Description: Typography for web dashboard
- 11 — Reference: MDN Web Docs — developer.mozilla.org, Description: HTML5/CSS3/JavaScript reference
- 12 — Reference: CSS Glassmorphism Design Pattern, Description: Modern UI design pattern for premium interfaces